

GitHub: PR abierta/actualizada

webhook pull_request (opened/synchronize),
firmado HMAC X-Hub-Signature-256 → POST
/api/webhooks/github

ASÍNCRONO: repo_is_authorized — installation_id →
VCSConnection.org → MonitoredRepo activo — encola
RQ y responde **202** al instante

Servidor Git: PR abierta/actualizada

cada git push que la sustenta dispara el hook
pre-receive (bash+curl) — Git no tiene un evento
de “PR” propio

SÍNCRONO: Bearer API key → require_scope +
ensure_api_key_repo_binding (clave atada 1:1 al repo)
— el git push queda bloqueado esperando

Token + datos reales de la PR

token efímero de instalación (JWT de App
firmado RS256 → access_token de 1h) →
get_pull_request_data + get_reputation_metadata
(siempre resuelta)

Payload del push

POST /api/v1/analyze con diff_text + metadata
(repo, ref_name, author_login extraído de git log)

NÚCLEO WATCHGATE (compartido)

QuotaService.analyze_with_quota → static · dependencies · vulnerabilities
· reputation · semantic (LLM, opcional) → agregación → score/semaforo

Comentario + evento live

client.post_comment en la PR (best-effort) +
actualización en tiempo real al Dashboard

Respuesta al script del hook

JSON {score, semaforo} — **sin comentario en la PR**,
solo stderr en la terminal de quien hizo el push

Aceptación humana en el Dashboard

el mantenedor ve el score y pulsa “Aceptar” (POST
/api/scores/{id}/accept)

Servidor Git: aplica el exit code

Forgejo · GitLab · Gitea · Gitolite · bare SSH — lee el
exit code que devolvió el hook pre-receive para ESA
ref

Merge real en GitHub

_try_merge_pull_request (token de instalación / PAT)
→ POST merge en la API de GitHub (pulls/{n}/merge)
 la PR se mergea de verdad — o falla (rama
protegida, checks, conflictos) y se reporta, sin romper
la aceptación ya registrada

 **exit 1 → RECHAZA** la
PR (la ref no se actualiza;
motivo por stderr)

 **exit 0 → ACEPTA** la PR
(la ref se actualiza; el
commit entra en el repo)